

约束理论（下）

≡ 标签

高效工作

≡ 简述

如何将其应用于知识型工作？

≡ 声明

仅供学习交流使用，非商业使用，..

≡ 备注

Tiago Forte|fortelabs.com

<https://flowus.cn/preview/3357cb05-ba90-43b5-8935-e0dd8af22455>

6. DBR 在微软的应用

在上一小节，我介绍了鼓-缓冲-绳（DBR）这一约束理论在制造业等生产环境中的应用。

这些听起来可能像是老掉牙的制造业套路，跟现在没啥关系。但别急着下结论，让我们来看一个真实案例，它把我

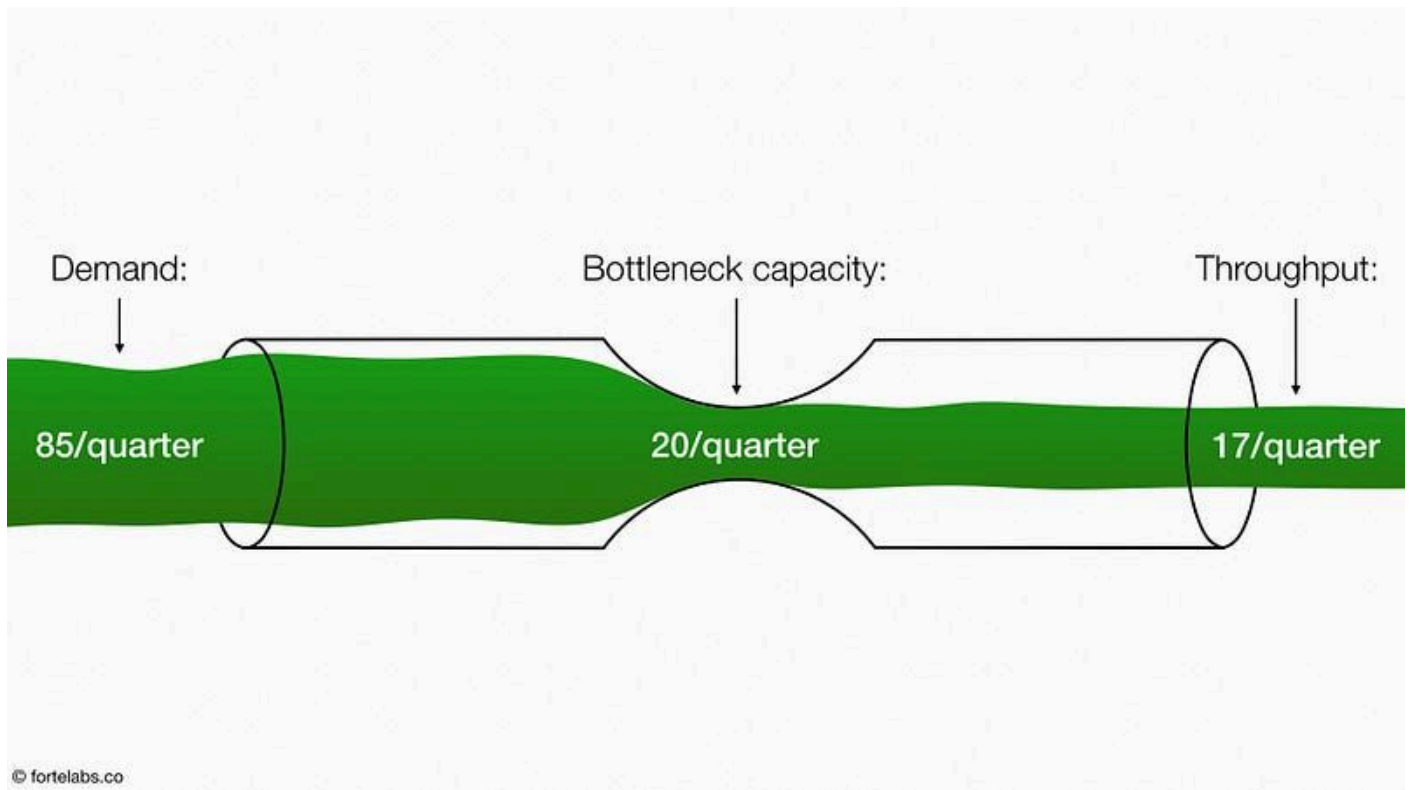
们这个系列至今讨论的所有理念都串联起来，而且发生在全球最具创新精神的公司之一：微软。

故事的起点是一个年轻的项目经理德拉戈斯·杜米特里乌，他决定接受一个挑战：运用约束理论的原则，来扭转公司八个 IT 部门中表现最差的软件开发团队。

九个月后，他成功了，这个团队变成了业务部门中表现最好的，生产力提升了 155%，交付周期从五个月缩短到两周，按时交付率从几乎为零提高到了 90% 以上。

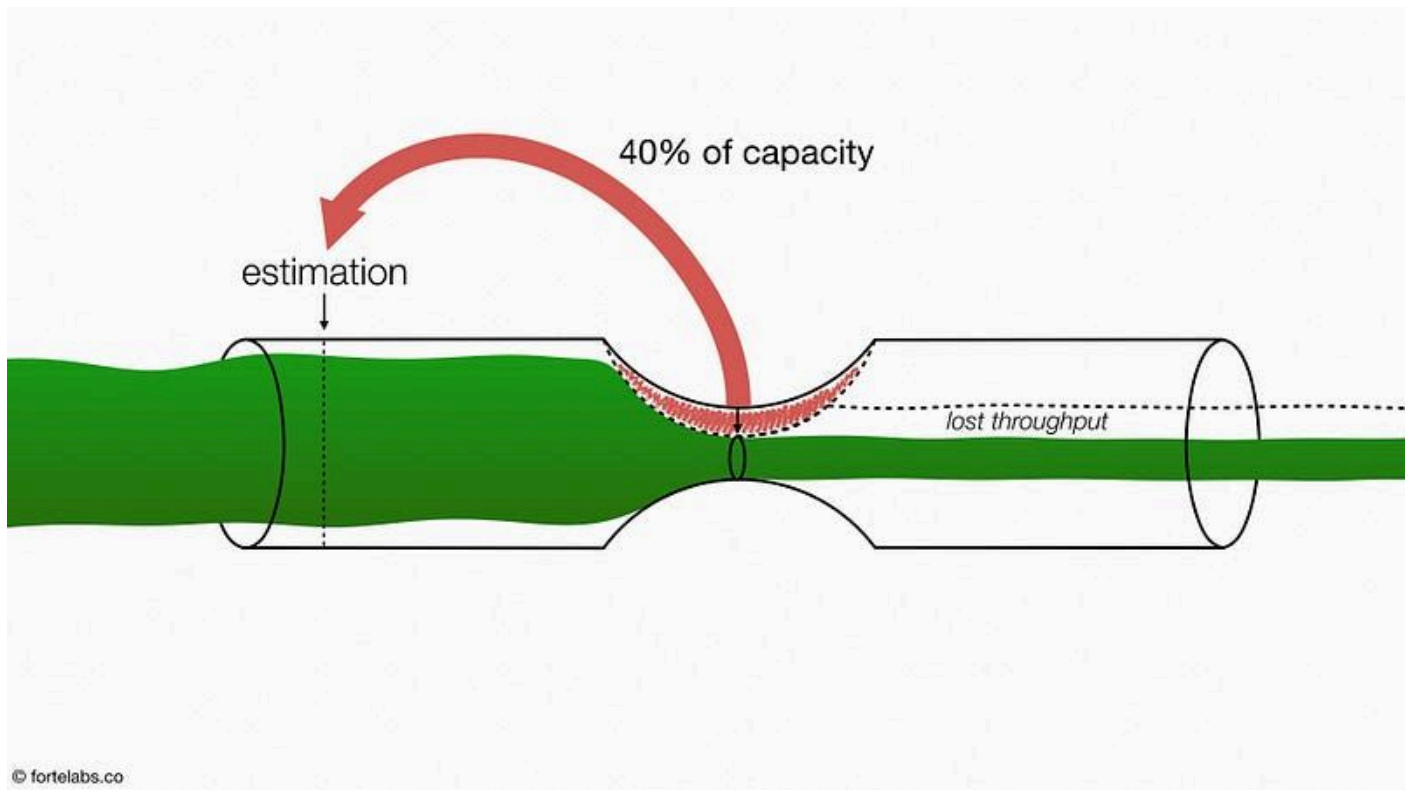
XIT 持续工程团队负责维护 80 多个供微软全球员工（或“客户”）内部使用的应用程序。这包括小型变更请求（通常是 bug 修复）的开发和测试。2005 年第一季度，这个团队的生产力跌到了历史最低点——积压工作量是产能的五倍（而且还在增长）。可想而知，需要维护应用程序的四个内部客户群并不满意。

XIT 通常每天收到大约一个变更请求，或者说每季度 85 个。但由三名开发人员组成的团队平均每人只能处理 6.5 个（总共约 20 个），导致上一季度实际完成量仅为 17 个请求。

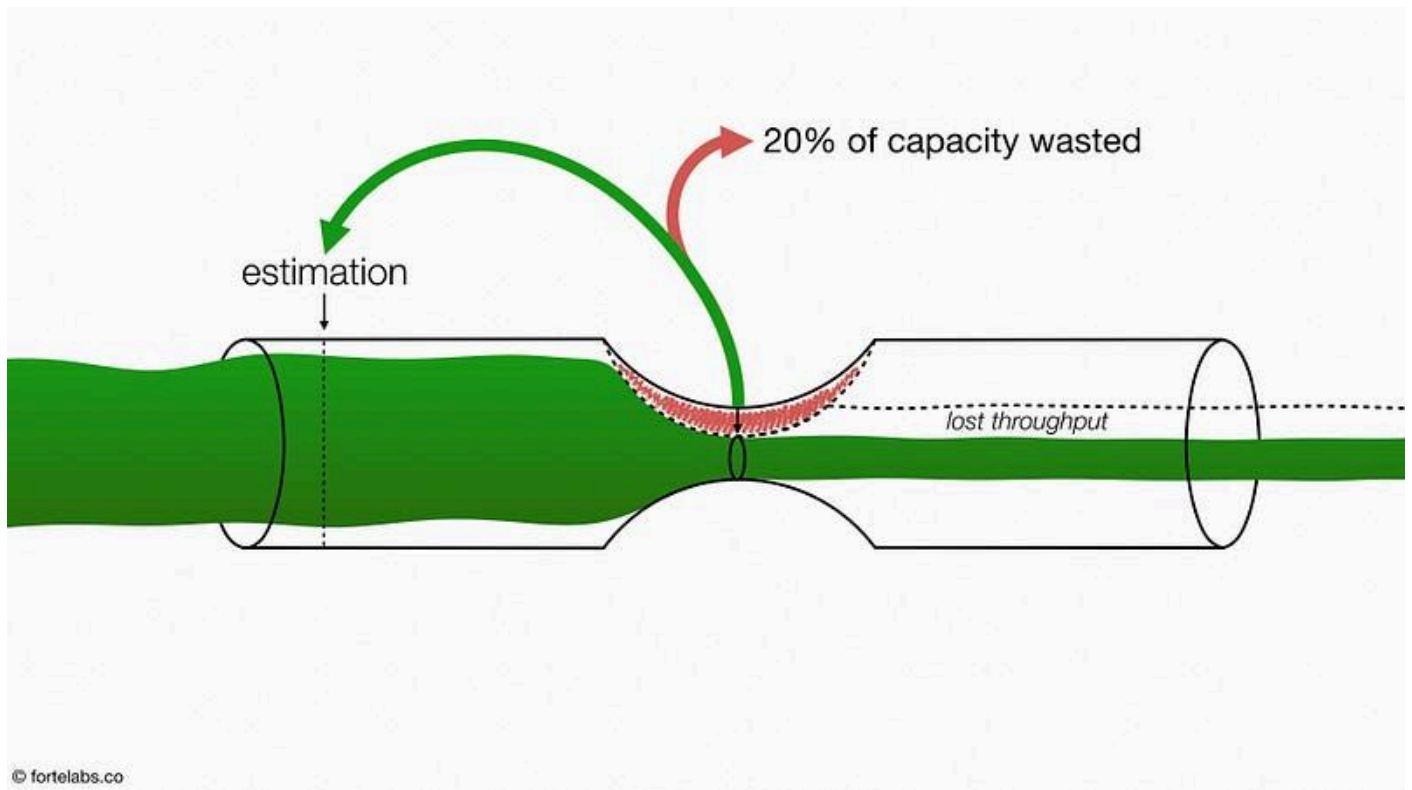


当新的变更请求到达时，需要对其进行评估以得出粗略的时间估算。XIT 与其内部客户之间的协议规定，这种估算必须在 48 小时内完成，这意味着它必须作为最高优先级加急处理。

这里我们已经可以看出问题的端倪——产生一个估算需要一名开发人员和一名测试人员各花费四个小时，这意味着每个变更请求会从系统中吸走一整天的生产力。每天一个请求，每个请求一天——他们原地踏步。仅仅是产生估算就占用了他们总产能的 40%。

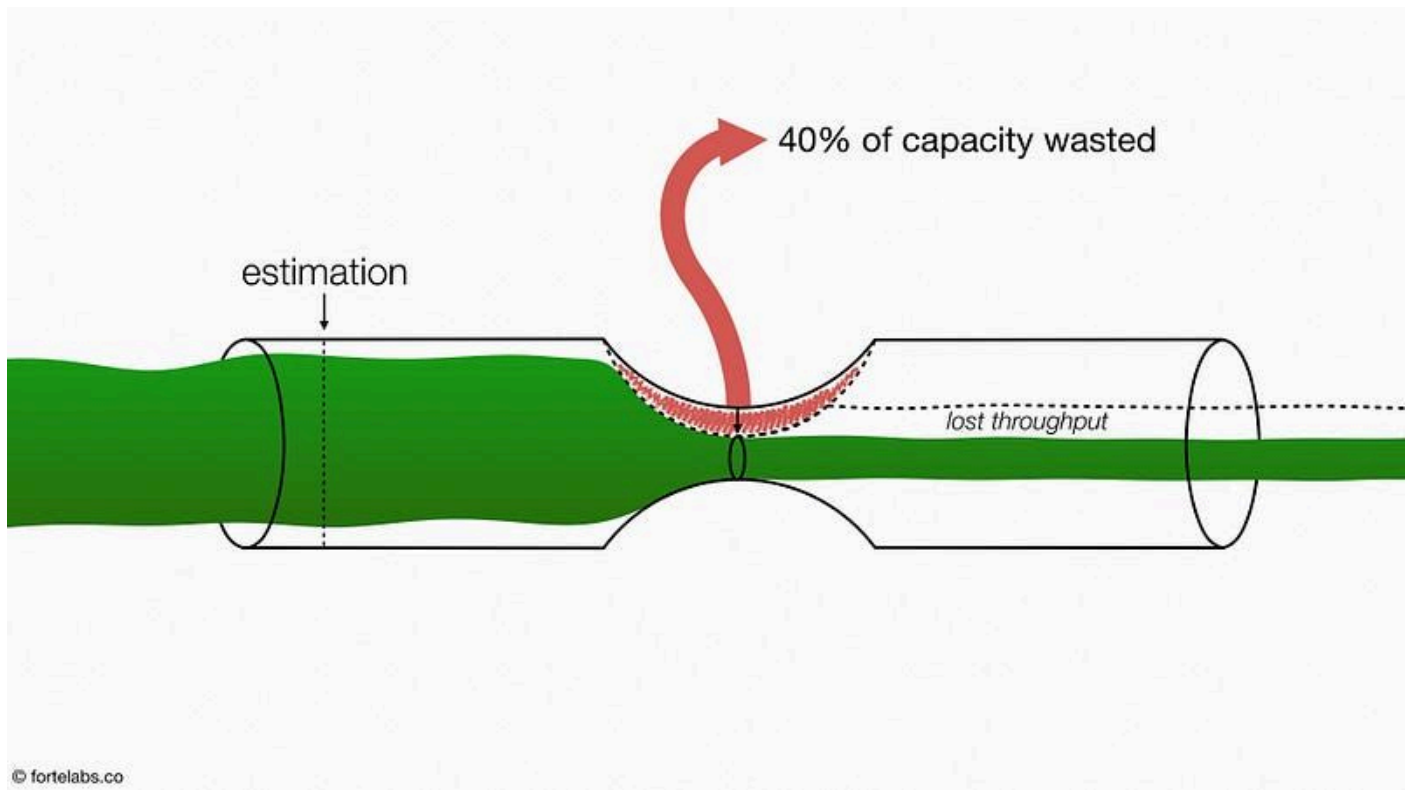


情况变得更糟。历史数据显示，团队完成的请求只有大约50%。其余的50%要么过于庞大，要么太昂贵，或者已经太迟。开发人员提供的估算中，只有一半实际上对产出有所贡献，这意味着团队的总产能中有15%到20%被浪费了。



更糟的是，他们完成的大部分评估工作永远不会被使用，因为等到开始工作时已经过去了太长时间，不得不重新进行。

这意味着几乎所有的评估工作都是浪费的。

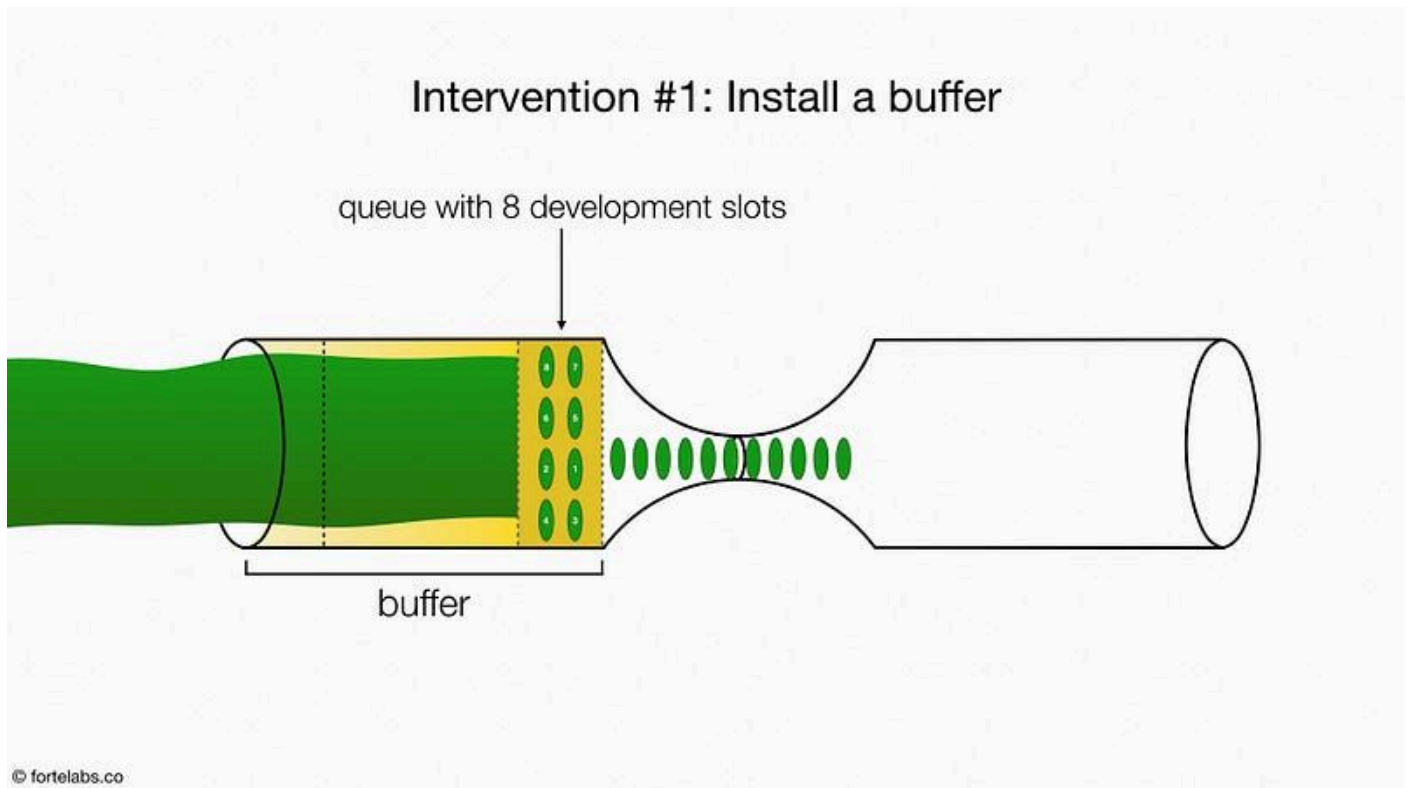


由于积压工作量如此之大，必须不断重新确定优先级。每月的优先级会议变得越来越紧张，客户开始争夺将他们的优先事项纳入其中。信任崩溃，因为他们不再相信他们的请求会在没有 1 级严重性紧急情况下得到处理。

听起来很熟悉吗？

杜米特里乌与大卫·安德森（最早将 TOC 应用于软件开发的顾问之一）合作，提出了三项干预措施。这些措施事后看来可能像是常识，但 TOC 的概念框架起到了信念强化和许可授予的作用，为团队工作方式和与客户沟通方式的微妙变化提供了支持。

第一项干预措施是在开发团队前面添加一个缓冲区。请记住，缓冲区就像一个“等候区”或队列，新的工作项在那里等待，直到准备好被处理。缓冲区的目的是阻止新请求直接流入瓶颈，在那里开发人员必须中断正在做的事情来处理它们。相反，缓冲区中的 8 个“槽位”将保留新请求，直到开发人员准备好审查它们。

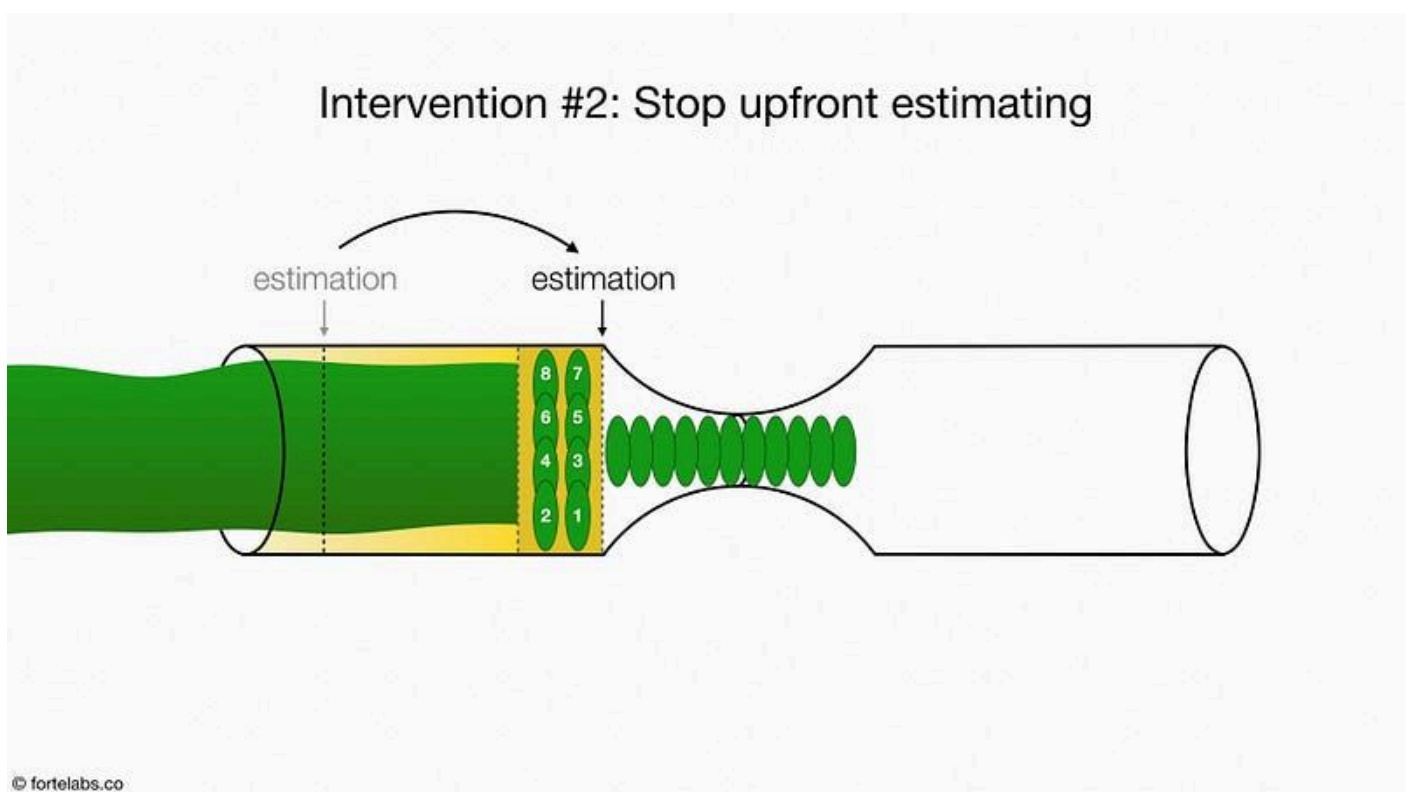


添加缓冲区产生了几十个积极效果：

- 项目经理只有在请求被估算、确定范围并分配到队列中的槽位后才会承诺交付日期，这使他们能够设定现实的期望和时间表

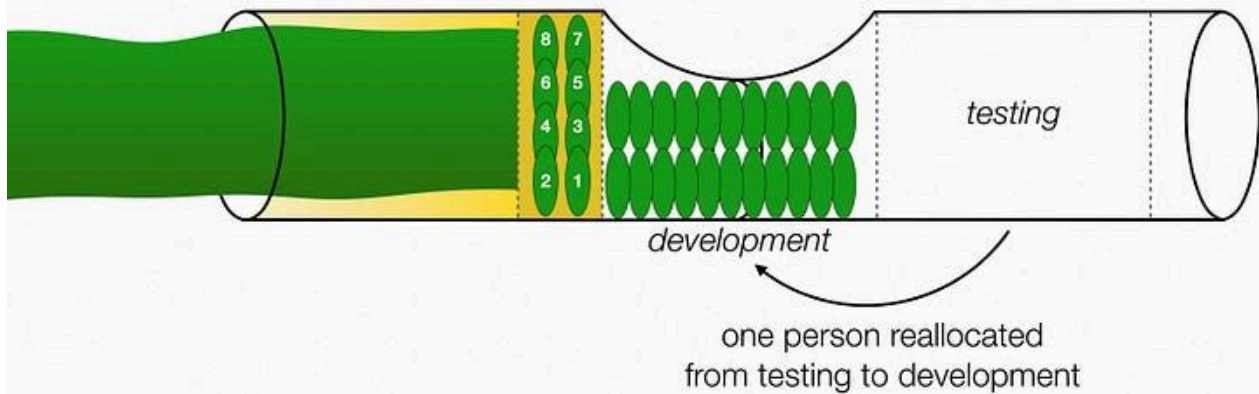
- 新请求的传入流被“扼制”到团队实际可以消化的水平，这减少了每个请求的交付时间，因为开发人员专注于单一任务，而不是在优先事项之间切换
- 每月的优先级会议变得不那么紧张，因为每个人都知道他们只需要填充 8 个新请求的“批次”

第二项干预措施是停止提供前期估算。每个变更请求都假定需要平均五天（对于边缘情况有几个额外规则）。这只有在缓冲区减少并稳定了每个请求所需的平均时间后才有可能。



第三项干预措施是重新分配资源以优化瓶颈。一个人从测试重新分配到开发，将比例从 3:3 改为 4:2 。

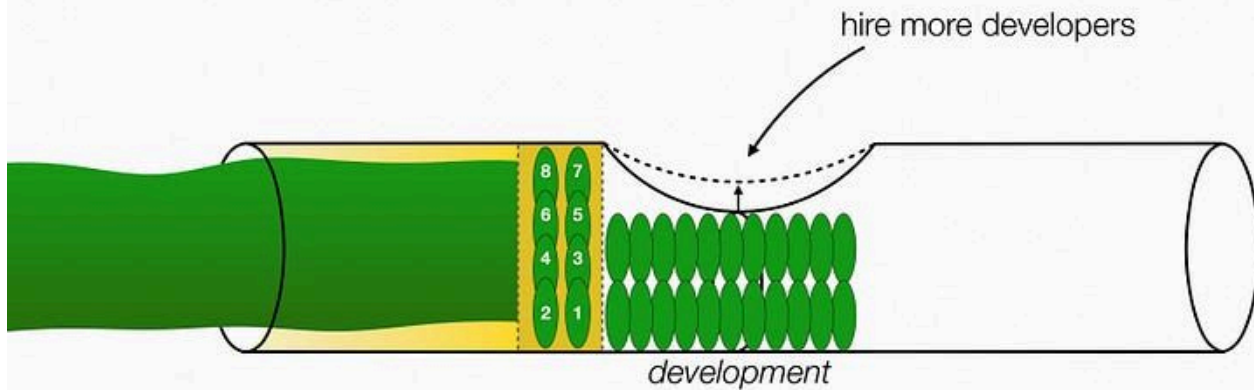
Intervention #3: Reallocate resources



© fortelabs.co

一旦瓶颈的产能通过现有资源得到了充分利用和扩展，就到了最后一步：通过招聘更多开发人员进一步增加瓶颈（从而增加系统）产能。需要注意的是，尽管增加瓶颈的产能非常重要，但在现有产能未被充分利用之前这样做是没有意义的。否则，就像在高速公路上加车道，但快车道却被一张沙发挡住了。

Final step: Increase bottleneck capacity



© fortelabs.co

XIT 持续工程团队的总产出从每季度 17 个请求增加到 56 个。积压工作从 80 个请求减少到不到 10 个，每个请求的平均成本从 7,500 美元降至 2,900 美元。客户再次感到满意。

人们常常认为，如果 TOC 适用于现代知识工作，那一定需要最先进的应用。在这个案例中使用像DBR这样的简单方法表明，限制软件工程师（以及其他知识工作者）生产力的大部分因素与他们如何执行工作的细节无关，而是与工作的管理、规划、调度和排队有关。

换句话说，工程部门可能是整个组织的瓶颈，但管理层却是提升工程部门的瓶颈。

7. 五个聚焦步骤

前面我们谈到微软的一个软件工程团队，他们用约束理论大幅提升了生产力。

但你可能心里犯嘀咕：他们怎么知道该改哪儿？他们是怎么制定出那套解决方案的？如果不清楚这些，我们顶多就是照搬一些死板的规则，这些规则在不同环境下可能根本不灵光。

现在，我们终于要深入到约束理论的精髓了，这是一种用于提升任何价值创造系统吞吐量的持续改进方法，叫做“五个聚焦步骤”：

第 1 步：找出约束

这一步告诉我们应该把改进的火力集中在哪儿，因为只有约束（也就是“瓶颈”）处的改进才能带来实质性的变化。

第 2 步：优化约束

在增加产能之前，我们得先充分利用现有的产能。这里的“优化”意味着“竭尽所能地发挥约束的最大效能”。

第 3步：使非瓶颈服从瓶颈

所有非约束的任务都得“服从”约束的需求，因为约束才是限制整个系统吞吐量的关键。

第 4步：提升约束

只有完成了前面的步骤，增加约束的产能才有意义，这样才能提升整个系统的性能。因为增加产能成本极高，所以我们把它作为最后的手段，而不是首选。

第 5步：回到第 1步

前四步的必然结果，也是这种方法被称为“持续”改进的原因，是约束会转移到其他地方。这一步让你重新回到起点。

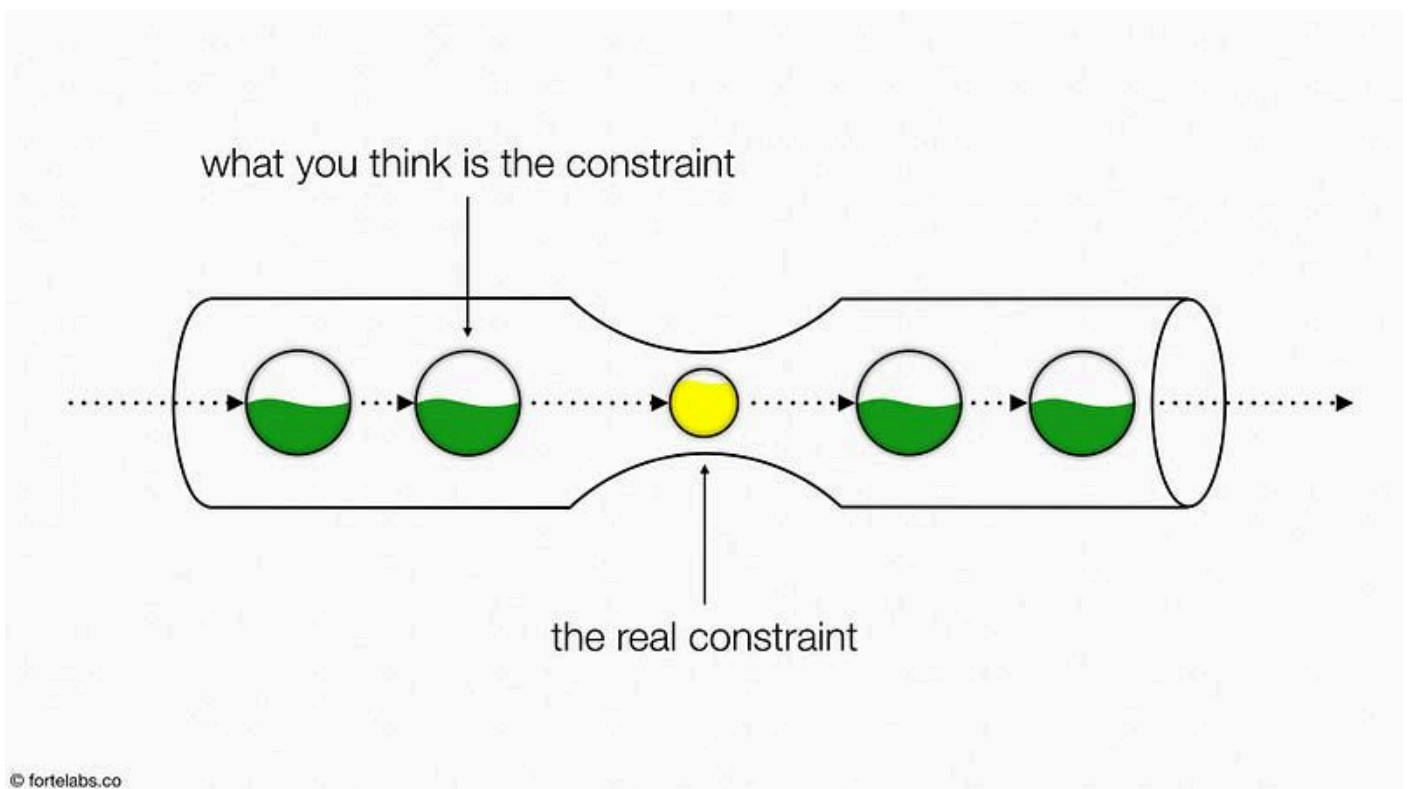
我们来仔细看看第 1步，以及为什么它代表了我们在组织内进行改进方式的重大转变。

识别约束

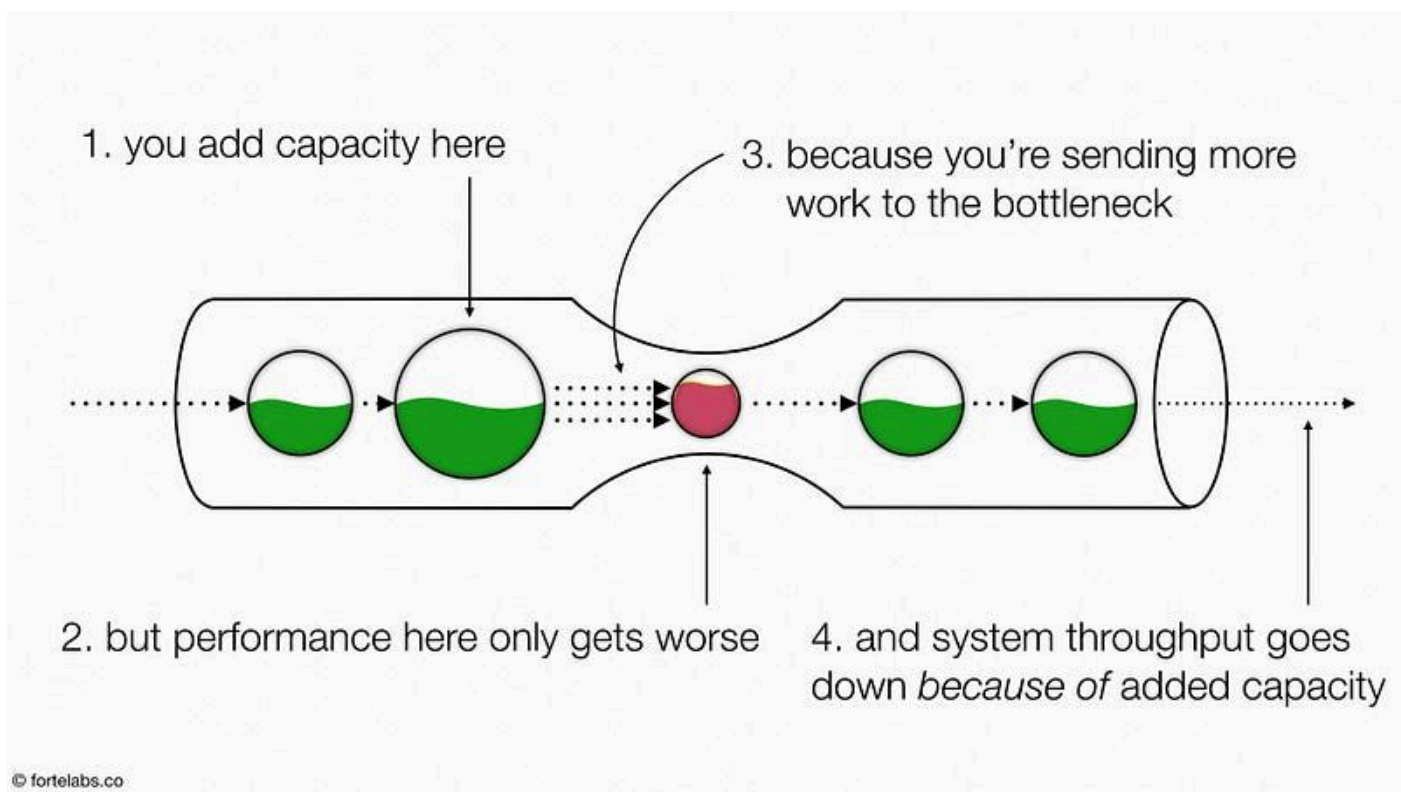
因为只有约束处的改进才能对整个公司产生影响，所以识别约束显然是第一步。

让人欣慰的是，你不必一开始就完美地识别出约束。这是一个自我纠正的过程，而不是瀑布式的过程，小错误不会滚雪球般变成大错误。与动态、相互关联的系统打交道的优势在于，它对变化的反应非常迅速。系统的行为会告诉你是否选对了。

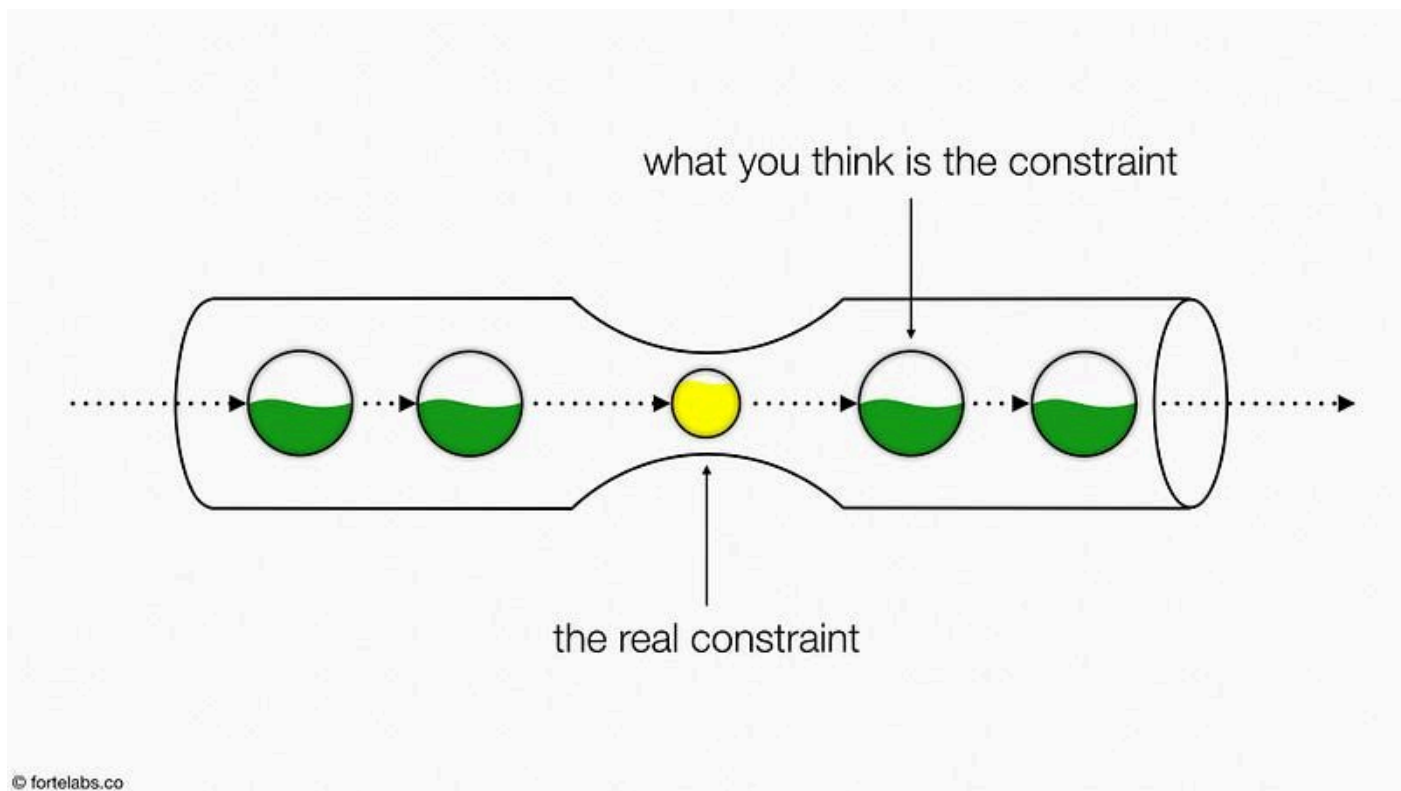
假设你错误地识别了约束，并在那里增加了产能，但实际的约束其实在下游：



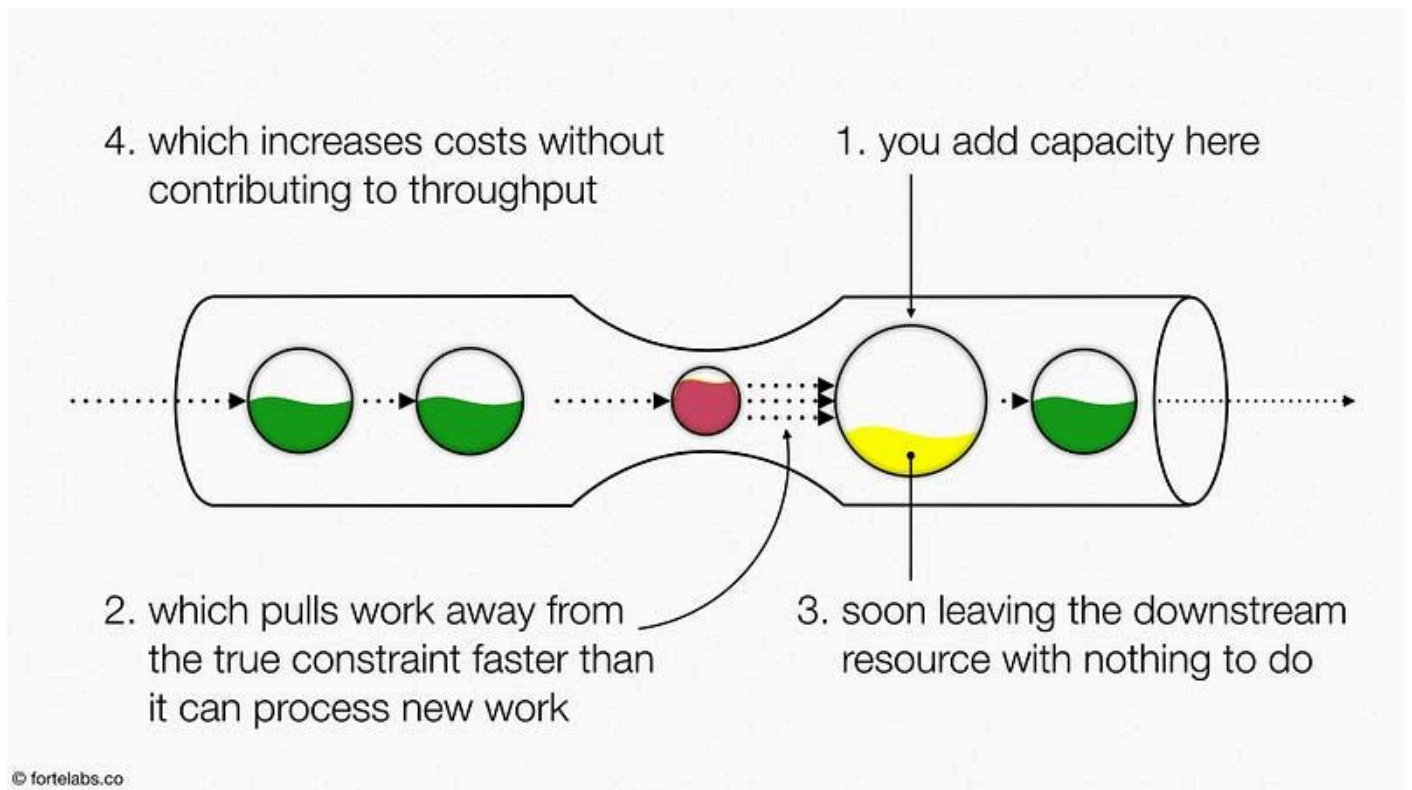
增加上游产能只会让真正的瓶颈承受更多工作，从而降低其吞吐量，进而降低系统的吞吐量。讽刺的是，系统性能的恶化不是因为你努力增加产能，而是因为增加了产能：



当你错误地将约束识别为真正约束的下游时，也会发生类似的情况：



增加更多的下游产能会比约束处理新工作更快地拉走已完成的工作。这意味着这个新扩展的部门很快就会缺乏可做的事情，增加成本却不会为吞吐量做出任何贡献：



通过这种方式，系统的行为会告诉你是否在改进正确的地方。而且它会很快告诉你，以吞吐量变化的形式在几天或几周内显现，而不是几个月或几年。

我们在这里讨论的实际上是对如何改进任何复杂系统（如公司）的一个根本性改变。

我们不再收集关于业务每个方面的大量数据，而是承认我们没有答案，只有假设。我们不再花大量时间制定涵盖每

种可能性的详细计划，而是对约束所在做出最佳猜测，让反馈成为我们的指导。

我们不再强制推行改变公司每个部分的全面性推广，而是将注意力集中在一个点上，即使我们不确定它是否正确。我们不再将业务不断变化的动态视为需要控制的敌人，基于某一时刻的快照做决策，而是鼓励系统循环得更快，以加速我们的反馈循环。

我们正试图培养德国人所说的 Fingerspitzengefühl，即“指尖感觉”——对情况不断变化的动态的直觉感知。就像在体育或战斗中一样，你是通过高频率地参与行动的中心来培养这种本能，而不是在办公室里制定计划。

延伸这个比喻，看看你组织的价值流图。如果你无法用手指指出你认为可能是约束的一个点，或者不愿冒险让这个假设被证明是错误的，你就永远无法集中精力。没有重点，你将只能在随机方向上启动几十个随机项目，希望通过运气或遵循规定来取得成功。即使你确实取得了成功，你也不会明白原因。

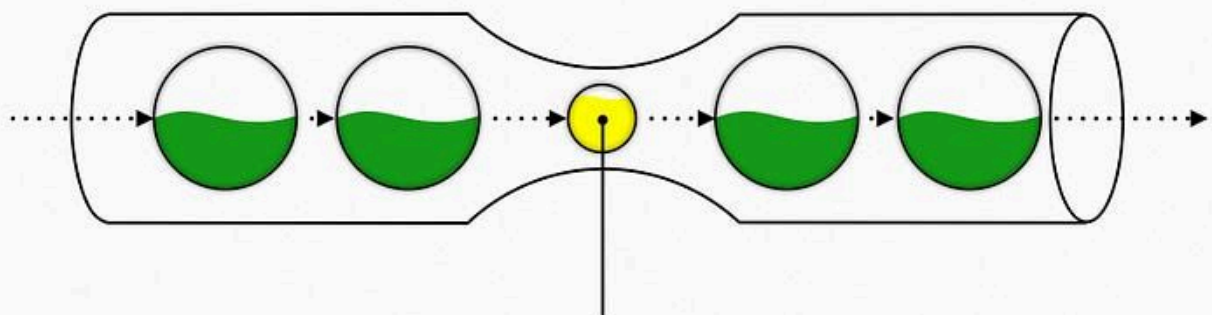
这就是为什么戈德拉特说，如果他必须用一个词来总结约束理论，那就是“聚焦”。

8. 找出瓶颈

前面，我介绍了五个聚焦步骤，这些步骤可以用来提升任何价值创造系统的效率：

1. 找出瓶颈
2. 优化瓶颈
3. 使非瓶颈服从瓶颈
4. 提升瓶颈
5. 回到第一步

但在实际的组织中，如何识别瓶颈呢？



#1 Identify the constraint

在工厂制造时代，这相对简单：走进工厂车间，看看哪里零件堆积如山（丰田称之为“现场”实践）。这也是为什么减少在制品库存如此重要——当工厂车间的材料最少时，等待某台机器完成操作的零件堆积就会非常明显。

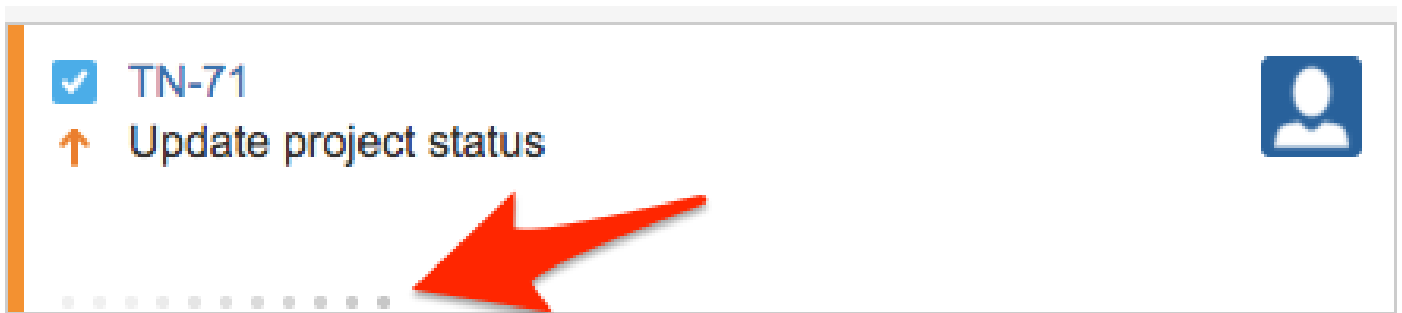
但在知识工作中，我们如何识别那些无形的瓶颈呢？这需要更多的技巧，但绝非不可能。

首先，你必须让在制品可视化。因为信息工作通常没有物理形态，所以现代生产力的很多做法都是从使其有形开始的：看板、甘特图、燃尽图、待办事项清单、日历和时间表，这些都是可视化工具，可以帮助我们更容易地评估工作量和进度。

其次，寻找最稀缺的资源。在个人层面，寻找日历或待办事项清单上反复推迟和延续的任务。这些往往是需要某些类型资源的任务——技能、心态、工具、时间块或他人在场——而这些资源供应短缺。例如，如果你个人的瓶颈是专注时间，那么需要这种时间的任务就会积累和延迟，这是许多人熟悉的经历。

在团队层面，看板上卡片减慢和堆积的满列是该步骤存在瓶颈的明显指标。团队任务管理软件（如Jira）可以跟踪

每个步骤的周期时间——即任务在该列中停留的平均时间。工作流程中的瓶颈通常是周期时间最长的步骤。



Jira通过在底部使用灰色小点来跟踪问题在某个列中停留了多少天。

第三，询问人们。以前两个步骤的可视化测量和数据收集为参考点，简单地问你的团队成员：你们最常等待的资源是什么？（提示：使用“最稀缺资源”而不是“瓶颈”这个词可以避免负面感受）

在产品开发流程的哪个点最可能出现“质量循环”——来回修改和返工。这些循环通常被归咎于“不确定性”或被视为“迭代”，代表工作在系统中向后移动，是标准化和流程改进的成熟目标。

通过这些问题，你要寻找的是那些反复出现的、阻碍人们前进的问题，导致他们“同时”开始新任务，从而使在制品

激增。常见问题包括需要其他部门的信息或决定、管理层的审查或批准，以及内部利益相关者的问题答复。

第四，寻找约束性政策和规则。在最初识别瓶颈的热情中，很容易立即着手解决看起来“最大最可怕”的问题。看起来大家抱怨最多的“最吱吱作响的轮子”必定能带来最大收益，这似乎很合乎逻辑。

但这通常是个错误。请记住上一篇文章中提到，关注瓶颈的全部意义不是发起一个庞大的、全面的改造项目。相反，我们从小小的、局部的改进开始，依靠系统的反馈循环来放大我们的努力，然后再加倍努力。

有趣的是，即使在制造业时代，戈德拉特也说，组织中最具限制性和破坏性的瓶颈几乎总是“软瓶颈”，如政策或规则，而不是机器或工作区设计等硬瓶颈。我们在第5部分中看到，一个看似无害的政策——在48小时内提供时间估算——导致一个软件工程团队浪费了40%的产能。

由于是无形和抽象的，政策可以在瞬间产生，只需主管的一句话或一支笔。它们甚至可能意外产生，源于误解的话语或不赞同的眼神。抽象规则不受物理限制或反压力的约束——一台极度失灵的机器很快就會在物理定律下崩溃，

而一个极度失灵的政策可能会扭曲员工的行为多年，才有人意识到发生了什么。指出这些政策的破坏性影响更加困难，因为它们只存在于人们的头脑中。

好消息是，它们的无形性也意味着约束性政策和规则是最容易改变的，并且效果最直接，因为它们也可以在瞬间消失。

第五，进行实验。完成前面的步骤后，你就可以进行一些实验了。你手头有测量吞吐量变化的方法（可视化在制品和绩效数据），以及对瓶颈可能在哪里的初步直觉（通过与员工的访谈和对规则和政策的敏感性）。

基本的实验是并发实验：列出公司或部门所有活跃项目的清单，划掉一半，严格禁止在进一步通知之前对不活跃的项目进行任何工作。

随着在制品的减少，瓶颈最少的工作中心将开始无事可做。这是一个关键时刻——你必须避免“给他们增加”新任务或将他们解雇为“多余产能”的诱惑。

当非瓶颈工作中心停止开始新项目时，他们会开始向瓶颈发送更少的工作，导致吞吐量上升。这是顿悟的时刻——当每个人工作更少时，公司反而产出更多。此时很容易说

服每个人，他们的工作是尽一切努力最大化瓶颈的生产力，而不是尽可能多地工作时间。

案例研究：看Tony Grout 和Chris Matts 讲述他们在 Skype 大规模应用 TOC 的情况（2000 名开发人员分布在 10 个地点）。他们发现，识别组织中的瓶颈是最重要的一步，因为它让问题对每个人都变得可见。

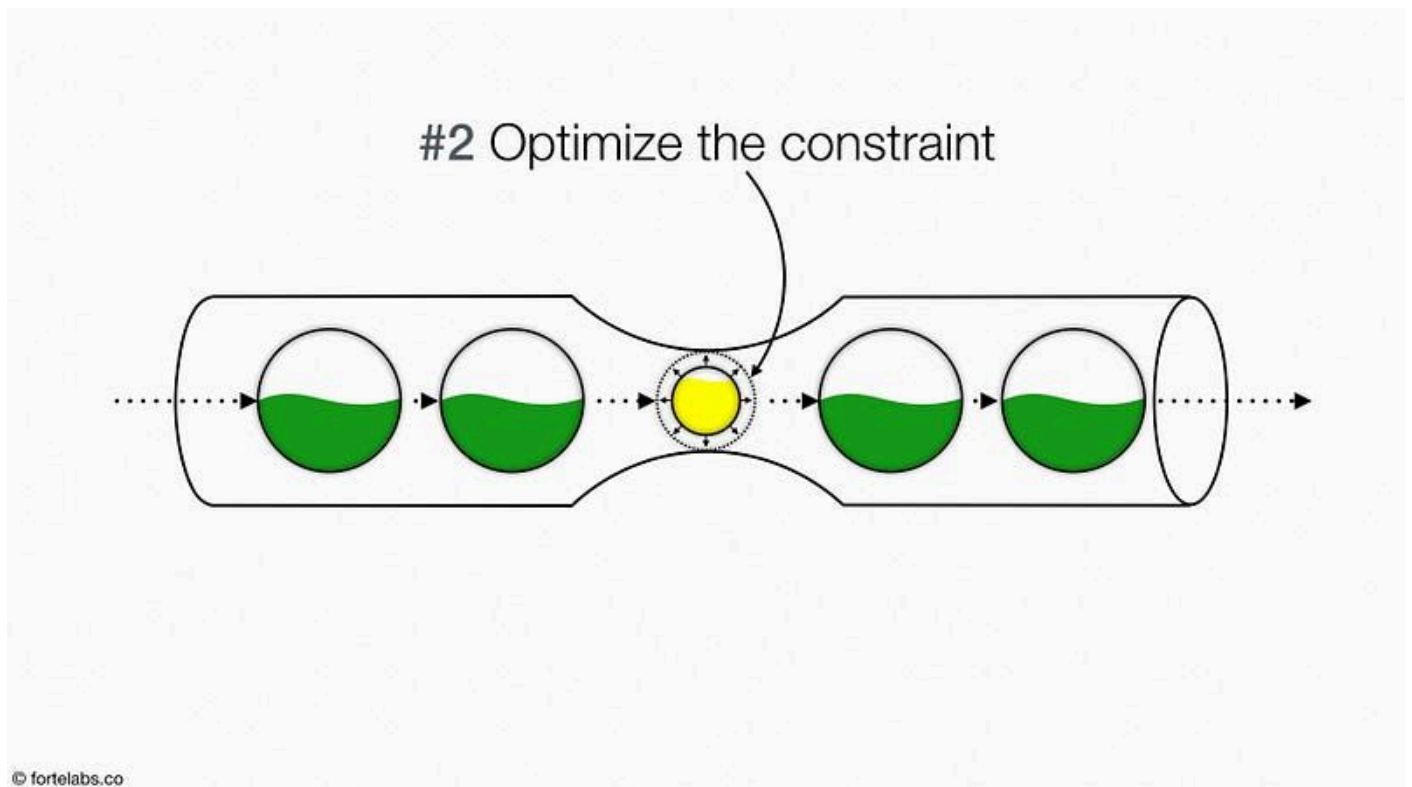


9. 优化瓶颈

前一部分，我讲述了如何在知识型组织中识别瓶颈。接下来，也就是五个聚焦步骤中的第二步，是对这个瓶颈进行

优化：

1. 找出瓶颈
2. 优化瓶颈
3. 使非瓶颈服从瓶颈
4. 提升瓶颈
5. 返回第 1 步



一旦你意识到在瓶颈处损失的每一分钟都是整个公司损失的一分钟，那么一系列之前看似太耗时、昂贵或风险太大的投资突然就变得有意义了。事实上，它们成为了你的首要任务。

以下是一份问题清单，可以启发你的思考：

缓冲：如何预先准备和打包工作单元，确保瓶颈永远不会闲置？

正如我们在之前的文章中看到的，这意味着瓶颈之前的人或团队必须有多余的产能，以便建立工作在制品的缓冲（或队列）。所有上游部门都应该花时间仔细打包他们的工作，以便瓶颈能轻松“消费”。

质量检查：如何在工作通过瓶颈之前检查其质量，确保不会在需要重做的任务上浪费时间？

一种做法是使用轻量级的原型来验证产品功能是否合理，避免在其上花费过多的工程时间。或者让编辑在将文案发送给网页设计师之前先检查一下，以避免耗时且容易出错的HTML编辑。

持续运作：如何确保瓶颈在尽可能多的可用时间内运行？

这可能意味着制定鼓励专注工作的政策，比如 Uber 创建了一项政策，规定戴着耳机的人不得被打扰。它也可以采取提供餐饮、公司班车或无会议日等形式。任何节省员工时间的做法都可能增加他们对瓶颈的贡献时间。

预防性维护：可以采取哪些预先行动来降低可能中断瓶颈的故障发生概率？

这包括物理维护、更新软件、检查服务器，以及备用电源或 WiFi。但在知识工作中，它还意味着预防情感和健康崩溃 - 提供福利和激励措施，鼓励员工锻炼、饮食健康、充足睡眠、照顾健康，甚至参与志愿服务和其他能让他们与更高目标联系的活动。

卸载：瓶颈当前执行的哪些任务可以由其他部门执行？

通常我们会将瓶颈视为一个人或一个团队，但实际上它通常是一种技能或能力。一个 10 人的软件开发团队可能只有一个人懂得特定的编程语言，或者了解整个系统架构。如果另一个开发人员也能执行这项任务 - 即使他们的效率远低于瓶颈 - 也值得从瓶颈那里卸载该任务。只要他们能承担哪怕 1% 的瓶颈工作，这项投资就开始以整个公司吞吐量的速度获得回报。

5S：如何组织瓶颈的工作环境以提高效率、效果和秩序？

借鉴自准时制生产，5S 指的是五类活动：整理、整顿、清扫、标准化和维持。对于知识工作来说，这可能包括从

整洁、视觉上令人愉悦且杂物最少的办公室；到为员工配备最新型号的电脑和用户友好的软件；再到标准化数字文件的命名、存储和更新方式。

安灯：如何让受瓶颈限制的资源更容易、更快地发现和解决问题？

工程师是否必须向庞大且反应迟钝的投诉管理系统提交工单？问题和障碍是不可避免的，所以不如擅长解决它们：快速响应的 IT 团队、通过自动售货机分发免费的计算机外设、通过实时仪表板显示瓶颈指标，并奖励员工花时间报告问题。安灯是另一种借鉴自准时制的技术，它实际上是创造一种将问题视为“值得珍惜的宝石”的文化，而不是需要隐藏或归咎于他人的错误。

让我们将上述所有内容结合成一个具体的例子。假设软件开发团队的瓶颈是一个具有特定技能的开发人员。如何充分优化他/她？

可以通过始终准备好一池可以开始的开发任务来保护她免于闲置（缓冲）。可以通过最小化沟通线路的公司结构来保护她免受干扰，通过安静的工作环境来避免分心（持续运作）。可以通过为她提供最好的设备和开发工具来进一

步优化（5S 和预防性维护）。她可以将进度报告和时间跟踪等任务卸载给支持人员或工具（卸载）。团队可以使用指导、结对编程、回顾和代码审查来确保代码中的错误被尽快发现和解决，或者更好的是，根本不引入错误（安灯）。她可以从清晰表述的产品需求中受益，以最大限度地减少她花在解释和澄清需求上的时间（质量检查）。

请注意，上述实施想法都不是新的。约束理论增加的是一个目标和实施系统：改进在哪里以及如何才能真正产生影响，以及如何将它们结合使用？

再举一个例子，考虑这个案例研究，肯塔基州的一家医院急诊室使用 TOC 来解决反复出现的问题并改善其可服务的患者数量。

经过两轮对医院员工的调查，瓶颈被确定为护士的可用性。他们有许多责任：为新入院患者准备房间、接待前台、处理病情，更不用说实际照顾患者了。由于他们有太多工作，护士经常无法为患者办理出院手续，患者等待造成了各个环节的积压。

他们制定的解决方案是雇用新员工专门负责一项简单但耗时的任务：整理床铺和准备房间。这通过让护士专注于更

受瓶颈限制的任务，提高了急诊室的吞吐量。它提高了护士（以及可能还有医生和患者）的士气，让他们能够专注于更能发挥其培训的高技能工作。值得注意的是，这个小变化（以及其他一些变化，如更好的标准操作程序）解决了调查中收集到的大部分员工投诉。

变革不必大、昂贵或戏剧性，但必须在正确的地方。问问自己：你在工作或生活中的改进努力是否在优化你的瓶颈？

如果不是，为什么还要费心呢？

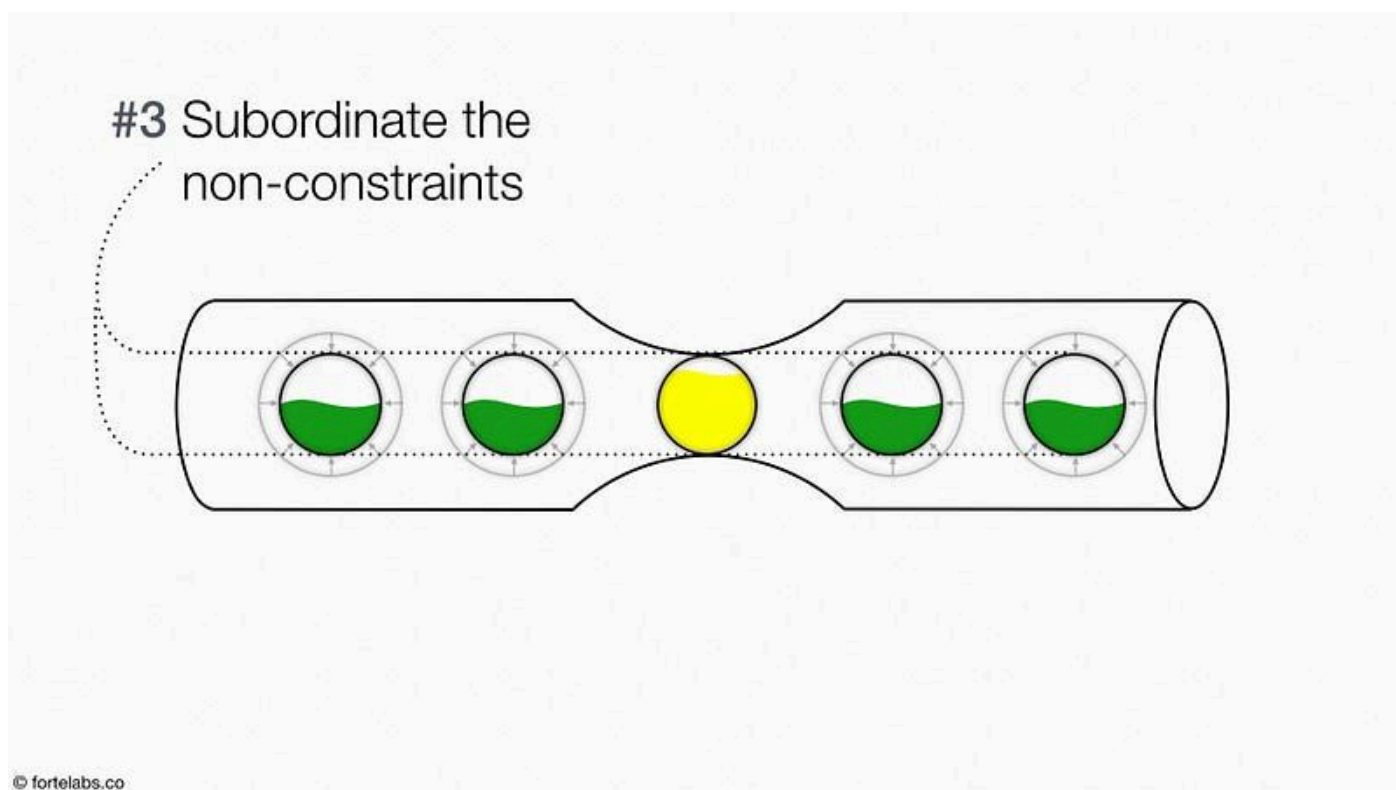
10. 让非约束服从约束

前面我讲述了如何优化组织中的瓶颈。下一步，即五个聚焦步骤中的第三步，是让所有其他员工的工作服从于这个瓶颈：

1. 识别瓶颈
2. 优化瓶颈
3. 让非瓶颈服从

4. 提升瓶颈

5. 返回第一步



这可能是五个聚焦步骤中最不被重视和最不直观的一步。要理解为什么服从如此关键，想想我们迄今为止讨论的几乎所有针对瓶颈的措施都不可能孤立进行。它们都需要非瓶颈的配合。

举个例子，为了确保瓶颈始终被充分利用，它需要始终有一队已包装并准备好开始的工作队列。这样一来，如果上游资源因为任何原因被中断或放慢，瓶颈就能继续不受干扰地处理队列中的工作——任务完成！

但这还不够。一旦上游资源恢复正常，它们需要快速补充已经消耗掉的队列。而它们能做到这一点的前提是有多余的（或冲刺）能力。正因为如此，为了让系统中的某个部分得到充分利用（瓶颈），其他部分必须具备多余的能力。

这个结论可能看起来有些反直觉：有时你必须先为非瓶颈部分创造多余的产能，然后才能增加瓶颈部分的产能！在我们之前看到的微软案例中，他们必须确保测试团队有足够的多余产能，然后才能尝试扩大作为瓶颈的开发团队。

在个人层面，在尝试通过增加“深度工作”时间来“扩展”瓶颈之前，首先要确保在其他方面有足够的缓冲——比如睡眠、锻炼、饮食和人际关系。你是否过着一种有足够缓冲的生活方式，以便能承载更多的专注时间？

但服从的真正有趣之处在于，它是政治和心理因素凸显的地方。谈论持续改进的崇高理念很好，直到你不得不指着某人告诉他们需要将自己的决策服从于别人的生产力。他们很可能会将此理解为“你的工作不那么重要。”

当然，我们每天在每个组织中都会做出服从决策，只是通常以文化上被认可和政治上安全的方式进行。我们发现很

容易“优先考虑”某些事务，并强调它们对组织目标的重要性。但我们常常忽略的是，每次优先考虑一项任务时，实际上我们也在隐性地降低了许多其他任务的优先级。这些是二阶决策效应，往往没有明确做出或意识到，其影响可能是微妙的，并且需要较长时间才能显现出来。

服从的另一个例子是**加急处理**。这是一个决定，将组织或团队所有精心计划的工作——当前项目、发布日期、优先需求、批次设置和商定的时间表——服从于一个危机。感觉上我们是在加速紧急请求通过常规步骤。实际上，我们是在减缓整个系统的速度，以释放所需的产能来做到这一点。

除了最不受重视、最不直观和最具政治性之外，服从也是最困难的一步。它直接与戈德拉特所描述的工作场所第一法则相对：保持忙碌。

为什么？因为服从看起来就像是除了一个资源外，所有资源都有空闲时间。这对所有相关人员的心理影响可能是压倒性的。几乎不可能不将空闲时间视为“自由”时间。

首先，因为我们没有真正的方法来衡量知识工作的生产力，我们用“填满的产能”作为替代指标。管理者努力确保

每个人总是有事可做。员工自己也确保保持忙碌，这使得管理者甚至难以判断真正的瓶颈在哪里。

其次，作为个人贡献者，我们所学到的几乎一切都鼓励我们优化自己的工作。我们按照自己喜欢的方式安排日程，打包可交付成果，尽可能清晰和严格地划定责任范围，并严格执行这些界限。这是另一件我们不愿听到的事情——我们不可能都按照自己喜欢的方式成功，同时让组织按照它需要的方式成功。

这就是为什么戈德拉特称成本会计为生产力的头号敌人：成本会计的设计是为了衡量每个单独资源的生产力，即每单位投入产出的量，而非整体的投入产出量。

由于运营费用（在知识型组织中，主要是人员工资）基本上是固定的，成本会计表明可以在不产生任何额外成本的情况下增加非瓶颈的产出。这似乎在孤立测量时提高了那个人的生产力。但正如我们现在所知，增加非瓶颈的产出不仅无法产生差异，实际上还可能通过用过剩的工作压垮瓶颈而降低整个系统的生产力。

这在知识工作中产生了更加有害的影响。在知识工作系统中流程的工作单元不是零件和原材料，而是高度易腐的想

法和需求。正如 David Andersen 在软件工程敏捷管理中解释的那样，每个想法和需求都隐含着围绕其含义和解释的知识体系。一旦不再被积极处理，这些知识就开始萎缩。最终，当人员离开时，它就完全丢失了。

当非瓶颈过度生产并在瓶颈前堆积大量知识工作库存时，就像杂货店供应商在冰箱外堆积大量冰淇淋桶，因为他们没有费心去了解冰箱的容量。当这些冰淇淋融化时（想法和需求过时），生产它们的全部成本（研究、分析、更新、审查、总结等）都必须作为损失注销。

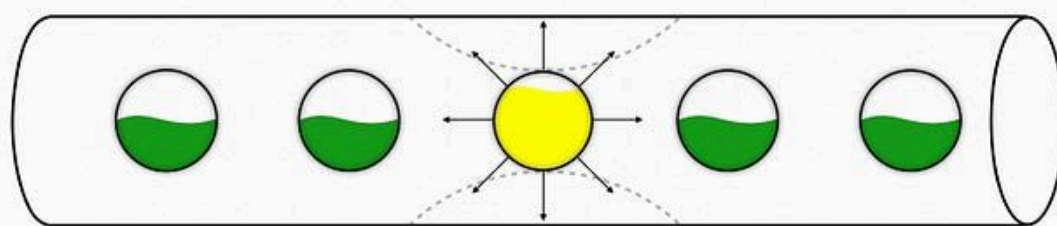
因此，忽视流程的原则并让每个人始终全力工作以“提高生产力”的极其讽刺的结果是，运营费用更高，产出却低于不做任何特殊努力的情况！

11. 提升瓶颈

上一部分，我讲述了如何让组织的非瓶颈因素服从于瓶颈，以最大化其产出。下一步，也就是五个聚焦步骤中的第四步，是提升（或缓解）瓶颈本身：

1. 找出瓶颈

2. 优化瓶颈
3. 使非瓶颈服从瓶颈
4. **提升瓶颈**
5. 返回第一步



#4 Elevate the constraint

© fortelabs.co

首先要注意的是，直到现在，在 9000 字的评论之后，我们才到达许多人用来“总结”约束理论的步骤：“哦，是的，我知道 TOC——它是关于改进瓶颈的。”

但是，当我们直接跳到第四步，忽略前三个步骤时，考虑一下我们错过了什么：

- 我们没有花时间正确地识别瓶颈，通过使在制品可视化，与最接近工作的员工交谈，寻找质量循环和重复依赖，或进行揭示瓶颈的实验。我们很可能会去解决“最吵闹的轮子”，这可能是最难改变的，而不是其他可能更容易和更有影响力的解决方案。
- 我们没有费心去优化我们已经拥有的能力，这意味着我们没有利用缓冲、预防性维护、工作流程简化或卸载。我们可能没有战略性地使用质量检查，也没有确保瓶颈尽可能多地工作。我们有可能花费大量时间和金钱来增加高速公路的新车道，而快车道却被一个巨大的沙发堵塞。
- 我们没有服从与非瓶颈，这对于实现任何收益都是必要的：确保非瓶颈有多余的冲刺能力，突出紧急加速的影响，并建立一种始终优先考虑瓶颈的文化。即使你对瓶颈所在的第一猜测恰好是正确的，任何改进也可能会被阻碍，因为其他人继续优化他们自己的工作，而牺牲你试图建立的任何变革。

缓解瓶颈只是旅程中的一步，而不是全部故事。

在《目标》附录的一次采访中，我们了解到南非比勒陀利亚大学医院的 Dr. Antoine Van Gelder 如何使用 TOC 来增加病人的吞吐量。

他们识别出他们的瓶颈是医生的时间供应，这可能并不令人惊讶。显然的解决方案似乎是雇佣更多的医生。这就是直接跳到第四步的样子。

通过更多的思考和观察，他们识别出一个特定的工作流程，占用了医生的大部分时间，而且也特别容易修复：人们不出席他们的预约。实际上，他们意识到他们正在使用他们最稀缺的资源——医生的时间——来执行一个低价值的功能：确认预约出席。

有了这个洞见，解决方案变得显而易见：他们开发了一个呼叫名单，他们称之为“病人缓冲”。护士被指派在病人预约的前一天或两天打电话，简单地询问他们是否计划出席。这实际上降低了未出席率，因为它起到了提醒的作用，即使他们不出席，也还有足够的时间找到替代者。

这个故事说明，事后看来解决方案很简单。但是，为了最大化你实际到达那里的机会，遵循这个过程是有帮助的。

当我们走到第四步，我们就有了一套工具来“提升”瓶颈或者优化其性能。以下是一些具体做法：

跟踪绩效数据

要对瓶颈环节的表现进行多维度、细致入微的跟踪，比起对所有工作中心的监控要更加深入。你的目标是找出并解决导致时间浪费的主要因素，而这需要跨职能团队的协作。

审核

在瓶颈之前、之中或之后加入特别的审核或质量检查，试图找出导致时间损失或质量缺陷的因素。请注意，你可以进行比平时更严格的审核，甚至是破坏产品单位的审核，因为你只在瓶颈处进行这些审核。精益生产中的“防错法”（*Poka-Yoke*）（顺便说一句，这绝对是最酷的日语精益术语）建议将缺陷检测和预防功能内置到设备中。现代等效的做法是在用户创建账户时弹出提示，建议他们选择更强的密码。

减少准备时间

丰田公司因此闻名于世（在那里被称为 SMED - 单分钟换模和 Jidoka - 部分自动化），这同样适用于知识工作：为软件程序员自动化部署、集成和测试，或为设计师使用模板或风格指南。

更新/升级

一旦你知道瓶颈是什么，你就可以花相当多的时间来制定其维护策略。这可能包括主动安排更新和升级，以避免不必要的停机时间（在精益生产中称为 TPM - 全面生产维护）。

增加能力

最后，你当然可以选择购买更多设备或在瓶颈环节雇佣更多人员。但需要注意，这种投入的经济逻辑与普通招聘完全不同。在瓶颈环节，即使聘请昂贵的专家也是值得的，因为瓶颈的每一单位产能都直接影响整个组织的吞吐量。由于增加产能的成本——无论是时间还是金钱——都非常高，因此这通常是最后的选择，而不是优先考虑的方案。

政策约束

正如我在第七小节中提到的，政策和规则往往代表着最具限制性的约束。由于其无形性，它们的影响是普遍的，但难以准确确定。这就是为什么第四步被称为“提升”

（elevating）或“缓解”（relieving）约束，而不是“扩大瓶颈” - 如果你对抗的是政策或规则，你希望减少而不是增加它！

范式约束

在约束理论所涉及的所有原则和技术中，对我来说最有趣的是范式约束。它们是那些使我们能够看到现实，但同时也限制我们视野的盲点和假设、世界观和过滤器。

即使在制造业中，这些范式也是一个重要因素，数十年的经验表明，即使在工厂里，人员管理也是至关重要的。

但在知识工作中，它们也变得至关重要。我们的挑战是将流程的普遍原则转化为大脑神秘的内部运作。要证明即使是虚拟的、非物质的、网络化的信息，而不仅仅是原材料，也可以实现产出的指数级提升。

当我们逐渐接近人类心理学领域时，越来越难分辨什么是约束，什么是自我。很明显，我们对“我们是谁”的概念本

身可能就是一种约束，而缓解这个最终的瓶颈需要的不仅仅是提升自我。